---

1. 스크립트 작성자 정보 및 툴 실행 환경 정의

tcl

```
# made by kjinsoo # module load #module load icc2/2021.06-SP5 # open
icc2_shell #icc2_shell -gui | tee -i ./03_powerplan.log
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 작성자(kjinsoo) 정보와 터미널에서 ICC2를 실행할 때 사용하는 리눅스 환경 설정(Module load) 및 로그 저장 명령어(`tee -i`)를 주석으로 기록해 둔 부분입니다.

---

2. 단계(Step) 정의 및 파일 입출력(I/O) 제어

tcl

```
#set current step & before step
set step powerplan
set before_step floorplan

#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#open lib
open_lib ../02_floorplan/nlib/ORCA_${before_step}.nlib

#copy lib
copy_lib -from ../02_floorplan/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib

#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib

#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
```

코드를 사용할 때는 주의가 필요합니다.

- **`set step... / set before_step...`**: 현재 단계인 `powerplan`과 이전 단계인 `floorplan`을 변수로 선언하여 경로 관리를 유연하게 만듭니다.

- **`source -e ...design_setup.tcl`:** 디자인 전체 환경 변수(멀티 코어 설정, 기술 파일 경로 등)가 담긴 tcl 파일을 실행(-e 옵션으로 에러 발생 시 즉시 중단)합니다.
- **`open_lib ... / copy_lib ...`:** 이전 플로어플랜 단계의 데이터베이스(`nlib`)를 안정적으로 보존하기 위해, 이를 현재 파워플랜 폴더로 복사(`copy_lib`)해 옵니다.
- **`close_lib -all / open_lib ... / open_block ...`:** 열려 있는 모든 라이브러리를 닫고 새롭게 생성된 파워플랜 라이브러리와 내부의 `ORCA.design` 블록(물리적 레이아웃 데이터)을 화면에 엽니다.

---

3. 기존 PG 제약 조건 및 배치 차단 구역 초기화

tcl

```
##############################
#remove constraints
##############################
remove_pg_strategies -all
remove_pg_patterns -all
remove_pg_regions -all
remove_pg_via_master_rules -all
remove_pg_strategy_via_rules -all
remove_placement_blockage [get_placement_blockages -filter
"blockage_type == hard"]
```

코드를 사용할 때는 주의가 필요합니다.

- **`remove_pg_* -all`:** 이전 단계에서 생성되었거나 찌꺼기로 남아있을 수 있는 전원(PG) 전략, 패턴, 영역 규칙, 비아 규칙을 전부 초기화하여 새로 작성할 스크립트와의 충돌을 방지합니다.
- **`remove_placement_blockage ... == hard`:** 플로어플랜 단계에서 설정되었던 물리적인 셀 배치 금지 구역(Hard Blockage)을 모두 제거하여 공간을 확보합니다.

---

4. 하드 매크로(Memory) 및 PLL 셀 변수 선언

tcl

```
##############################
#set macro & PLL full name
##############################
#all Macros & PLL
```

```
set all_macros [get_cells -hierarchical -filter "is_hard_macro &&
!is_physical_only"]

#only Macros
set only_macros "I_ORCA_TOP/I_CONTEXT_MEM/CONTEXT_RAM0
I_ORCA_TOP/I_CONTEXT_MEM/CONTEXT_RAM1
I_ORCA_TOP/I_CONTEXT_MEM/CONTEXT_RAM2
I_ORCA_TOP/I_CONTEXT_MEM/CONTEXT_RAM3
I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_B_RAM
I_ORCA_TOP/I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM
I_ORCA_TOP/I_PCI_READ_FIFO/PCI_RFIFO_RAM
I_ORCA_TOP/I_PCI_WRITE_FIFO/PCI_WFIFO_RAM
I_ORCA_TOP/I_SDRAM_READ_FIFO/SD_RFIFO_RAM
I_ORCA_TOP/I_SDRAM_WRITE_FIFO/SD_WFIFO_RAM"

#only PLL
set pll "I_CLOCK_GEN/I_PLL_PCI I_CLOCK_GEN/I_PLL_SD"
```

코드를 사용할 때는 주의가 필요합니다.

- **`set all_macros ...`**: 디자인 내부의 모든 물리적/논리적 하드 매크로(RAM 등)를 찾아 변수에 저장합니다.
- **`set only_macros "..."`**: 전원 공급을 집중적으로 제어할 핵심 메모리 블록(RAM 10개)의 인스턴스 경로 명을 명시적으로 지정합니다.
- **`set pll "..."`**: 아날로그 전원을 사용하여 별도 관리가 필요한 PLL(클럭 생성기) 2개의 경로를 변수에 지정합니다.

---

5. PLL 및 매크로 주변 배치 차단 구역(Hard Blockage) 1차 설정

tcl

```
#######################################
#create_placement_blockage_for PLL
#######################################
set pll_bbox [get_attr [get_cells $pll] bbox]
set i 0
foreach x $pll_bbox {
    set llx [expr [lindex $x 0 0] + 0.5]
    set urx [expr [lindex $x 1 0] - 0.5]
    set lly [expr [lindex $x 0 1] + 0.5]
    set ury [expr [lindex $x 1 1] - 0.5]
    set new_bboxes "{$llx $lly} {$urx $ury}"
    create_placement_blockage -name "pll_blkg_hard_${i}" -type hard
-boundary $new_bboxes
    incr i
}
```

코드를 사용할 때는 주의가 필요합니다.

- **`get_attr ... bbox`**: PLL 셀들의 절대 좌표(Bounding Box: 좌하단, 우상단 좌표)를 가져옵니다.
- **`foreach x $pll_bbox { ... }`**: PLL들의 좌표를 하나씩 돌면서 처리합니다.
- **`expr ... + 0.5 / - 0.5`**: 툴이 자동으로 전원선을 깔 공간을 마련하기 위해, PLL의 원래 크기보다 안쪽으로 `0.5미크론`씩 줄어든 가상의 영역(`new_bboxes`)을 계산합니다.
- **`create_placement_blockage ... -type hard`**: 계산된 영역 내부에 스탠다드 셀이 침범하지 못하도록 `pll_blkg_hard_0, 1`이라는 이름의 배치 차단 구역을 만듭니다.

tcl

```
#######################################
#create_placement_blockage_for macros
```

코드를 사용할 때는 주의가 필요합니다.

*(위의 **PLL** 블록 구조와 완벽히 동일한 메커니즘으로 메모리 매크로 **10**개(`only_macros`) 주변에 안쪽으로 **0.5**미크론 축소된 하드 블로케이지 `mc_blkg_hard_X`를 순차적으로 생성합니다.)*

---

6. Core Ring (칩 전체 외곽 전원 링) 생성

tcl

```
##############################
#create_core_Ring
##############################
#set pg via_master_rule
set_pg_via_master_rule pgvia_10x10 -via_array_dimension {10 10}

#create_ring_pattern
create_pg_ring_pattern core_ring_pattern -horizontal_layer M7 \
  -horizontal_width {5} -horizontal_spacing {2.188} \
  -vertical_layer M6 -vertical_width {4} -vertical_spacing {5.12} \
  -corner_bridge false \
  -via_rule { {intersection: adjacent} {via_master : pgvia_10x10} }

#set_pg_strategy
set_pg_strategy core_ring \
  -core \
  -pattern {{name: core_ring_pattern} \
```

```
  {nets: {VDD VSS VDD VSS}} {offset: {10 10}}}

#compile_pg
compile_pg -strategies {core_ring} -tag {CORE_RING}
```

코드를 사용할 때는 주의가 필요합니다.

- **`set_pg_via_master_rule ... {10 10}`**: 가로세로 교차점에 10행 10열 짜리 거대한 비아 배열(Via Array)을 뚫을 규칙(`pgvia_10x10`)을 정의합니다.
- **`create_pg_ring_pattern ...`**: 수평선은 메탈 7(두께 5, 간격 2.188), 수직선은 메탈 6(두께 4, 간격 5.12)을 사용하는 사각형 링 구조의 패턴을 만듭니다.
- **`set_pg_strategy ...`**: 위의 패턴을 코어 전체 영역(`-core`)에 적용하며, 선 배치는 외부 경계로부터 `10미크론 오프셋`을 두고 `VDD-VSS-VDD-VSS` 순서로 복수 링을 돌리겠다고 전략을 세웁니다.
- **`compile_pg ...`**: 설정한 `core_ring` 전략을 기반으로 실제 메탈 도선을 레이아웃에 실제 물리적으로 렌더링**(**생성**)**합니다. 이 구조물에는 `CORE_RING`이라는 태그가 붙습니다.

---

7. Macro Ring (메모리 매크로 주변 전원 링) 생성

tcl

```
####################################
#create Macro Ring
####################################
#create_Ring_pattern
create_pg_ring_pattern Macro_ring -horizontal_layer M5
-horizontal_width {1} -horizontal_spacing {0.16} -vertical_layer M6
-vertical_width {1} -vertical_spacing {0.16} -corner_bridge false

set_pg_strategy ORCA_macro_ring -macros $only_macros -pattern {
{pattern: Macro_ring} {nets: {VSS VDD}} {offset: {0.5 0.5}} }

#create_via_rule
set_pg_strategy_via_rule M_ring_vias -via_rule { \
  {{{strategies: {ORCA_macro_ring}} {layers: {M5}}} {existing: {strap
}}{via_master: {default}}} \
  {{{strategies: {ORCA_macro_ring}} {layers: {M6}}} {existing: {strap
}}{via_master: {default}}} \
}

#compile_pg
compile_pg -strategies {ORCA_macro_ring} -via_rule {M_ring_vias} -tag
{MACRO_RING}
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_pg_ring_pattern ...`**: 수평 M5(두께 1), 수직 M6(두께 1)로 이루어진 소형 링 패턴을 정의합니다.
- **`set_pg_strategy ... -macros $only_macros`**: 10개의 메모리 매크로 테두리에 딱 맞춰 `VSS-VDD` 전원 링을 오프셋 0.5를 두고 감싸도록 전략을 세웁니다.
- **`set_pg_strategy_via_rule ...`**: 새로 생길 매크로 링(M5, M6)이 기존에 존재하거나 앞으로 생길 다른 전원선(Strap)과 교차할 때 자동으로 기본 비아(`default`)를 뚫어 연결하도록 규칙을 만듭니다.
- **`compile_pg ...`**: 매크로 전원 링을 물리적으로 생성합니다. (`MACRO_RING` 태그 부여)

---

8. Macro Ring Connection (매크로 링과 내부 핀 연결)

tcl

```
##################################
#create Macro Ring connection
##################################
#create_macro_Ring_conn_pattern(vias)
create_pg_macro_conn_pattern Macro_ring_pin -pin_conn_type
scattered_pin -layers {M5 M6}

set_pg_strategy Macro_pins_via -macros $only_macros -pattern {
{pattern: Macro_ring_pin} {nets: {VSS VDD}} }

#compile_pg
compile_pg -strategies {Macro_pins_via} -tag {Macro_RING_con}
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_pg_macro_conn_pattern ...`**: 매크로 내부의 전원 핀들이 군데군데 흩어져 있는 형태(`scattered_pin`)일 때, M5 및 M6 레이어를 통해 외곽 링과 연결해 주는 패턴을 정의합니다.
- **`set_pg_strategy ...`**: 10개 메모리 매크로의 VSS/VDD 핀을 링과 결합시키는 전략을 정의하고, `compile_pg`로 물리적인 비아 및 연결선을 생성합니다.

---

9. PLL 용 독립 전원 링 및 연결선 생성 (PCI PLL & SD PLL)

tcl

```
####################################
#create Ring for PLL_PCI
####################################
create_pg_ring_pattern PLL_PCI_ring -horizontal_layer M5
-horizontal_width {1} -horizontal_spacing {0.16} -vertical_layer M4
-vertical_width {1} -vertical_spacing {0.16} -corner_bridge false

set_pg_strategy ORCA_PLL_PCI_ring -macros "I_CLOCK_GEN/I_PLL_PCI"
-pattern { {pattern: PLL_PCI_ring} {nets: {AVSS DVDD AVDD}} {offset:
{0.5 0.5}} }

set_pg_strategy_via_rule PLL_PCI_ring_vias -via_rule { \
  {{{strategies: {ORCA_PLL_PCI_ring}} {layers: {M5}}} {existing: {strap
}}{via_master: {default}}} \
  {{{strategies: {ORCA_PLL_PCI_ring}} {layers: {M4}}} {existing: {strap
}}{via_master: {default}}} \
}

compile_pg -strategies {ORCA_PLL_PCI_ring} -via_rule
{PLL_PCI_ring_vias} -tag {PLL_PCI_RING}
```

코드를 사용할 때는 주의가 필요합니다.

- 특이사항**:** PLL은 노이즈에 극도로 민감하므로 일반 디지털 전원(VDD/VSS)과 분리된 아날로그 전원 네트**(**`AVSS`**,** `DVDD`, `AVDD`**)**를 3중 구조로 인가합니다. 수평 M5, 수직 M4를 사용해 PCI PLL 주위에 전용 링을 형성합니다.

tcl

```
################################
#create_PLL Ring connection
################################
create_pg_macro_conn_pattern pll_ring_pin -pin_conn_type scattered_pin
-layers {M5 M6}
set_pg_strategy pll_pins_via -macros "I_CLOCK_GEN/I_PLL_PCI" -pattern {
{pattern: pll_ring_pin} {nets: {AVSS DVDD AVDD}} }
compile_pg -strategies {pll_pins_via} -tag {PLL_PCI_RING_con}
```

코드를 사용할 때는 주의가 필요합니다.

- PCI PLL 내부 전원 핀을 방금 만든 3중 아날로그 링에 M5/M6 레이어를 이용해 수동 형태로 매핑/연결시킵니다.

*(이후 나오는 `create Ring for PLL_SD` 및 `create_PLL Ring connection` 섹션은 대상만 **SDRAM 용 PLL(`I_CLOCK_GEN/I_PLL_SD`)*로 바뀐 것이며 완전히 동일한 아날로그 *3*중 링 구조를 생성합니다.*)*

---

10. 외부 수동 라우팅 스크립트 로드

tcl

```
################################
#PLL Power Manual Routing
################################
source ./user_script/PLL_power_manual_routing.tcl

################################
#create_Power_PAD_connection stripe
################################
source ./user_script/PAD_manual_connection.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- 자동 툴 기능만으로는 정교하게 처리하기 어려운 **PLL** 전원선 수동 라우팅과 외부 패드**(PAD)**와 전원 스트랩을 직접 이어주는 커스텀 결선 스크립트를 외부에서 가져와 실행합니다.

---

11. Power Mesh (칩 전체를 덮는 M8 / M7 고전력 그물망) 생성

tcl

```
################################
#create_Power_MESH
################################
#create placement blockage before create MESH78 - because of PLL manual
routing
set MESH_blk_bbox "{{1014.6780 716.0880} {1189.4520 737.6880}}
{{1014.6780 662.1780} {1189.4520 683.7780}}"
set i 0
foreach x $MESH_blk_bbox {
    create_placement_blockage -name "MESH_pblk_${i}" -type hard
-boundary $x
    incr i
}
```

코드를 사용할 때는 주의가 필요합니다.

- **`set MESH_blk_bbox ...`:** 방금 전 수동으로 라우팅한 PLL 전원선들의 좌표 영역을 지정합니다. 메인 그물망(Mesh)이 이 위를 지나가다가 쇼트(Short)를

내지 않도록 일시적으로 차단 구역(`MESH_pblk_0, 1`)을 지정하는 영리한 테크닉입니다.

tcl

```
#set M8 MESH boundary
set M8_MESH_bdry "{250.5480 214.4400} {1189.4520 1225.5600}"

#create_pg_reigion
create_pg_region MESH -polygon $M8_MESH_bdry
```

코드를 사용할 때는 주의가 필요합니다.

- 메탈 8 그물망이 생성될 구체적인 중심 좌표 범위를 지정하고, 이 영역을 `MESH`라는 이름의 전원 생성 리전으로 선언합니다.

tcl

```
#create_M8_mesh_pattern
create_pg_mesh_pattern METAL8_PG_PATTERN -layers {{vertical_layer: M8}
\
  {width: 4.64} \
  {spacing: 5.088} \
  {pitch: 19.456} \
  {trim: false}} \
  -parameters {4.64 19.456 5.088}

#creat_M8_pg_stretegy
set_pg_strategy METAL8_MAPPING_STRATEGY \
  -pg_regions MESH \
  -pattern {\
  {name: METAL8_PG_PATTERN} \
  {nets: {VDD VSS}} \
  {parameters: {4.64 19.456 5.088}} \
  {offset_start: {0 0}}} \
  -blockage {placement_blockages: all}

#create_M8_via_rule
set_pg_strategy_via_rule M8_via \
  -via_rule {via_master: NIL}

compile_pg -strategies {METAL8_MAPPING_STRATEGY} -via_rule {M8_via}
-tag MESH_8
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_pg_mesh_pattern ...`**: 수직 방향 최상위 메탈 8(두께 4.64, 선간 간격 5.088, 중심간 거리 19.456)의 굵은 전원 배선 구조를 선언합니다.

- **`set_pg_strategy ... -blockage {placement_blockages: all}`**: 방금 전 설정한 PLL 보호 구역을 포함하여 모든 블로케이지 영역을 우회(Skip)하도록 전략을 수립합니다.
- **`via_rule ... via_master: NIL`**: 이 단계에서는 아래 레이어와의 자동 비아 연결을 수행하지 않고(`NIL`) 오직 순수하게 M8 메탈 선만 깔도록 통제합니다. (비아는 추후 수동 제어 예정)
- **`compile_pg ...`**: 칩 상단에 강력한 M8 VDD/VSS 수직 메래 격자망을 렌더링합니다. (`MESH_8` 태그)

tcl

```
#create_M7_mesh_pattern
create_pg_mesh_pattern METAL7_PG_PATTERN -layers {{horizontal_layer:
M7} \
  {width: 1.104} \
  {spacing: 5.584} {pitch: 13.376} \
  {trim: false}} \
  -parameters {1.104 13.376 5.584}

#create_M7_MESH_strategy
set_pg_strategy METAL7_MAPPING_STRATEGY \
  -core \
  -pattern {\
  {name: METAL7_PG_PATTERN} \
  {nets: {VDD VSS}} \
  {parameters: {1.104 13.376 5.584}} \
  {offset_start: {0 0}}} \
  -blockage {placement_blockages: all}

#create_M7_via_rule
set_pg_strategy_via_rule M7_via \
  -via_rule {via_master: NIL}

#compile_pg
compile_pg -strategies {METAL7_MAPPING_STRATEGY} -via_rule {M7_via}
-tag MESH_7

#remove placement blockages if you create MESH78 done
remove_placement_blockages [get_placement_blockages -filter "full_name
=~ MESH_pblk_*"]
```

코드를 사용할 때는 주의가 필요합니다.

- 메탈 7 레이어를 이용해 수평 방향 격자선(두께 1.104, 피치 13.376)을 코어 전체에 생성합니다. (`MESH_7` 태그)
- M8/M7 격자망 구축이 모두 안전하게 끝났으므로, 수동 라우팅 보호용으로 쳐두었던 임시 배치 차단 구역(`MESH_pblk_*`)을 깨끗하게 지워 공간을 환원합니다.

---

12. Low Mesh (하위 메탈 M2 전원 격자망) 생성

tcl

```
########################
#create_M2 Low MESH
########################
#create_M2_mesh_pattern
create_pg_mesh_pattern P_m2_triple \
  -layers { \
  { {vertical_layer: M2} \
  {width: 0.44 0.192 0.192} \
  {spacing: 2.724 3.456} \
  {pitch: 9.728} {offset: 1.216} {trim : false} } \
  }

#create_M2_pg_strategy
set_pg_strategy S_m2_vddvss \
  -core \
  -pattern { {name: P_m2_triple} {nets: {VDD VSS VSS}} {offset_start:
{250.548 0}} } \
  -blockage { {{nets: VDD VSS} {placement_blockages : all}} }

#create_M2_via_rule
set_pg_strategy_via_rule M2_via \
  -via_rule {via_master: NIL}

#compile_pg
compile_pg -strategies {S_m2_vddvss} -tag {low_MESH_2} -via_rule
{M2_via} -drc no_check
```

코드를 사용할 때는 주의가 필요합니다.

- **`width: 0.44 0.192 0.192`** / **`nets: {VDD VSS VSS}`**: 하위 레이어인 M2 메탈을 사용해 3줄이 한 쌍으로 움직이는 독특한 수직 구조(`Triple`)를 만듭니다. VDD선은 0.44미크론으로 두껍게, VSS선 두 개는 0.192미크론으로 얇게 쪼개어 배치합니다.
- 스탠다드 셀 바로 위 영역이므로 DRC 검사를 잠시 끄고(`-drc no_check`) 고속으로 물리 선만 생성합니다. (`low_MESH_2` 태그)

---

13. Standard Cell Rail (M1 레이어 실제 셀 전원 레일) 생성

tcl

```
######################
#create_STD_RAIL
######################
#set via master rule for connect M1 to M2
set_pg_via_master_rule pgvia_1x2 -via_array_dimension {2 1}

#create_M1_std_cell_conn_pattern
create_pg_std_cell_conn_pattern std_rail_pattern -layers M1 -rail_width
{0.06} -rail_shift {3.344} -parameters {M1 0.06 3.344}

#create_M1_pg_strategy
set_pg_strategy M1_std_cell_rail \
  -core \
  -blockage {{macros_with_keepout: all}{placement_blockages: all}} \
  -pattern {{pattern: std_rail_pattern} {nets: {VDD VSS}} {parameters:
{M1 0.06 3.344}}} \
  -extension {{stop: core_boundary} {direction: L B R T }}

#create_M1_via_rule
set_pg_strategy_via_rule M2_via_stdcellrail \
  -via_rule { \
  { {strategies: {M1_std_cell_rail}} \
  {{existing: strap}{layers: M2}} \
  {via_master: pgvia_1x2} {between_parallel: true} } \
  { {intersection: undefined}{via_master: NIL} }}

#compile_pg
compile_pg -strategies {M1_std_cell_rail} -via_rule
{M2_via_stdcellrail} -tag {STD_RAIL_M1}
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_pg_std_cell_conn_pattern ...`:** 실제 배치될 수만 개의 스탠다드 셀들의 전원 핀에 직접 맞닿을 초미세 M1 수평 레일(두께 0.06미크론)을 정의합니다.
- **`-extension { ... }`:** 전원 공급 레일이 중간에 끊기지 않고 좌/우/상/하 코어 경계 끝까지 쭉 연장되어 닿도록 설정합니다.
- **`set_pg_strategy_via_rule ... pgvia_1x2`:** 새로 생성되는 M1 레일이 바로 위층에 깔려있던 M2 로우 메쉬(`low_MESH_2`)와 교차하는 지점마다 **2**행 **1**열짜리 비아(**`pgvia_1x2`**)를 생성하여 두 레이어를 촘촘하게 묶어줍니다. (`STD_RAIL_M1` 태그)

---

14. 모든 자동 생성 전략 클리어

tcl

```
######################
#remove Settings
######################
remove_pg_strategies -all
remove_pg_patterns -all
remove_pg_regions -all
remove_pg_via_master_rules -all
remove_pg_strategy_via_rules -all
```

코드를 사용할 때는 주의가 필요합니다.

- 주요 메탈 라인과 격자 생성이 완료되었으므로, 불필요해진 컴파일용 제약 조건 및 자동 규칙들을 다시 한번 전부 삭제합니다. 이제부터 정밀 제어를 위한 수동 비아 삽입 단계로 진입하기 때문입니다.

---

15. DRC01 수정을 위한 수동 대형 비아(Via Master) 매핑

tcl

```
#######################################
#change & create via for fixing DRC01
#######################################
#create CoreRing M7 to M8 via
set_pg_via_master_rule pgvia_10x10 -via_array_dimension {10 10} ;#set
via_master_rule
set core_M7_M8_shape [get_shapes -filter "tag == CORE_RING &&
layer.name == M7 || tag == MESH_8"] ;#set CoreRing(M7) MESH(M8) shapes
& selection
change_selection $core_M7_M8_shape
create_pg_vias -nets {VDD VSS} -to_types {lib_cell_pin_connect stripe}
-from_layers M7 \
  -to_layers M8 -via_masters {pgvia_10x10} -tag {core_M7_8_via} -shapes
[get_selection] ;#create vias for CoreRing(M7) to MESH(M8)
```

코드를 사용할 때는 주의가 필요합니다.

- **`get_shapes -filter ...`**: 이전에 만들었던 외곽 코어 링(`CORE_RING` 중 M7 레이어)과 최상위 메쉬(`MESH_8`) 도선들의 교차 도형 오브젝트를 수집합니다.
- **`create_pg_vias ... -via_masters {pgvia_10x10}`**: 툴이 간혹 놓치거나 에러를 내는 대전류 교차점에 **10x10** 크기의 초대형 비아 뭉치를 강제로 직접 박아 넣어 완벽한 전기적 결합을 이끌어냅니다.

tcl

```
#create MacroRing M6 to M7 via
```

```
set_pg_via_master_rule pgvia_3x3 -via_array_dimension {3 3} ;#set
via_master_rule
set M6_M7_shape [get_shapes -filter "tag == MACRO_RING && layer.name ==
M6 || tag == MESH_7"] ;#set MacroRing(M6) MESH(M7) shapes & selection
change_selection $M6_M7_shape
create_pg_vias -nets {VDD VSS} -to_types {lib_cell_pin_connect stripe}
-from_layers M6 \
  -to_layers M7 -via_masters {pgvia_3x3} -tag {M6_7_via} -shapes
[get_selection] ;#create vias for MacroRing(M6) to MESH(M7)
```

코드를 사용할 때는 주의가 필요합니다.

- 매크로 링(M6)과 전원 메쉬(M7)가 교차하는 부위에 3x3 비아 어레이를 강제로 뚫어 연결합니다.

tcl

```
#create M7 to M8 via
set_pg_via_master_rule pgvia_5x8 -via_array_dimension {8 5} ;#set
via_master_rule
set M7_M8_shape [get_shapes -filter "tag == MESH_8 || tag == MESH_7"]
;#set MESH(M7) MESH(M8) shapes & selection
change_selection $M7_M8_shape
create_pg_vias -nets {VDD VSS} -to_types {lib_cell_pin_connect stripe}
-from_layers M7 \
  -to_layers M8 -via_masters {pgvia_5x8} -tag {M7_8_via} -shapes
[get_selection] ;#create vias for MacroRing(M7) to MESH(M8)
change_selection
```

코드를 사용할 때는 주의가 필요합니다.

- 메인 메탈 격자인 M7과 M8이 교차하는 모든 격자점마다 8행 5열 짜리 비아 어레이(`pgvia_5x8`)를 촘촘히 박아 칩 전역의 저항(IR Drop)을 최소화합니다. 이후 선택 영역을 초기화(`change_selection`)합니다.

---

16. DRC02 수정을 위한 하위 비아 속성 변경 및 최적화

tcl

```
########################################
#change & create via for fixing DRC02
########################################
#change via def M1 to M2 vias
set M1_M2_via [get_vias -filter "tag == STD_RAIL_M1"] ;#set StdRail(M1)
LowMesh(M2) vias & selection
change_selection $M1_M2_via
```

```
set_attribute -objects [get_selection] -name via_def -value
[get_via_defs -library [current_lib] VIA12SQ_C] ;#change via defs for
StdRail(M1) to LowMesh(M2)
change_selection
```

코드를 사용할 때는 주의가 필요합니다.

- **`set_attribute ... -name via_def`:** 앞서 생성된 M1 레일과 M2 로우 메쉬 사이의 비아들을 선택한 뒤, 공정(Foundry) 라이브러리에 정의된 더 안정적이고 정밀한 스퀘어 타입 비아**(`VIA12SQ_C`)**로 일괄 강제 변경하여 비아 불량을 예방합니다.

tcl

```
#create M2 to M7 vias
set_pg_via_master_rule pgvia_2x4 -via_array_dimension {2 4}
set M2_M7_shape [get_shapes -filter "tag == low_MESH_2 || tag ==
MESH_7"] ;#set LowMesh(M2) MESH(M7) shapes & selection
change_selection $M2_M7_shape
create_pg_vias -nets {VDD VSS} -to_types {lib_cell_pin_connect stripe}
-from_layers M2 \
  -to_layers M7 -via_masters {pgvia_2x4} -tag {M2_7_via} -shapes
[get_selection] ;#create vias for LowMesh(M2) to MESH(M7)

#change M2 to M7 vias columns
set ch_M2_via [get_vias -filter "tag == M2_7_via && owner.name == VSS"]
;#set LowMesh(M2) MESH(M7) vias & selection
change_selection $ch_M2_via
set_attribute -objects [get_selection] -name number_of_columns -value 1
;#change via coluns for LowMesh(M2) to MESH(M7)
change_selection
```

코드를 사용할 때는 주의가 필요합니다.

- 하위 M2 메쉬와 상위 M7 메쉬가 교차하는 곳에 2x4 비아를 생성합니다.
- 이때 **VSS(**접지**)** 선의 경우, M2 레이어 패턴 특성상 공간이 협소하여 쇼트나 밀도 위반(DRC)이 발생할 수 있으므로 비아의 열 개수(`number_of_columns`)를 **1**개로 줄여 크기를 최적화시킵니다.

---

17. 최종 배치 차단 구역(Hard Blockage) 마진 확장 업데이트 (2차)

tcl

```
#remove placement_blockage
```

```
remove_placement_blockage [get_placement_blockages -filter
"blockage_type == hard"]

########################################
#create_placement_blockage_for macros
########################################
set macro_bbox [get_attr [get_cells $only_macros] bbox]
set i 0
foreach x $macro_bbox {
    set llx [expr [lindex $x 0 0] - 2]
    set urx [expr [lindex $x 1 0] + 2]
    set lly [expr [lindex $x 0 1] - 2]
    set ury [expr [lindex $x 1 1] + 2]
    set new_bboxes "{$llx $lly} {$urx $ury}"
    create_placement_blockage -name "mc_blkg_hard_${i}" -type hard
-boundary $new_bboxes
    incr i
}
```

코드를 사용할 때는 주의가 필요합니다.

- **`expr ... - 2 / + 2`:** 앞서 생성했던 밀착형 블로케이지(0.5미크론 축소형)를 완전히 지우고, 이번에는 매크로 외곽 경계선보다 **2**미크론 더 넓게 확장된 강력한 차단 구역을 재생성합니다.
- 전원선 라우팅이 완전히 끝났으므로, 다음 단계인 물리적 셀 배치(Placement) 단계에서 매크로 주변에 스탠다드 셀이 너무 바짝 붙어 타이밍 노이즈나 혼잡도(Congestion) 문제를 일으키는 것을 원천 차단하기 위한 마진 확보 작업입니다.

tcl

```
#####################################
#create_placement_blockage_for PLL
```

코드를 사용할 때는 주의가 필요합니다.

(PLL 주변 구역 역시 동일하게 사방 *2*미크론씩 확장된 대형 하드 배치 차단 구역을 재지정합니다*.)*

---

18. 파워 플랜 디자인 검증 및 데이터베이스 저장 후 종료

tcl

```
################
#check_Powerplan
```

```
################
#Drc check
check_pg_drc

#missing via check
check_pg_missing_via

#connectivity check
check_pg_connectivity ;# default
check_pg_connectivity -nets {VDD VSS AVSS DVDD VDDIO VSSIO}
-check_std_cell_pins none ;# check pg_net & except std cells

#save & exit
save_lib -all
exit
```

코드를 사용할 때는 주의가 필요합니다.

- **`check_pg_drc`:** 전원 배선의 이격 거리(Spacing), 두께(Width) 등 파워플랜 전용 디자인 룰 위반 여부를 검사합니다.
- **`check_pg_missing_via`:** 상하 레이어 교차점 중 비아가 누락되어 전원이 끊긴 곳(Open)이 없는지 추적합니다.
- **`check_pg_connectivity`:** 명시된 핵심 전원/접지 네트들(`VDD`, `VSS`, 아날로그 `AVSS/DVDD`, 입출력 `VDDIO/VSSIO`)이 끊김 없이 완벽히 연결되었는지 확인하되, 스탠다드 셀 핀 검사는 제외(`none`)하여 검사 속도를 높입니다.
- **`save_lib -all / exit`:** 검증 결과에 문제가 없다면 현재까지 작업한 모든 레이아웃 셀과 라이브러리를 안전하게 디스크에 저장하고 ICC2 툴을 종료합니다.